Compte rendu de TP

ListView de contacts modifiable

Kotlin - ListView - ArrayAdapter - EditText - notifyDataSetChanged()

Etudiante : Hanine Mjahed

Date : 19 octobre 2026

Depot GitHub : https://github.com/haninemjahed-lgtm/ContactsListView

1. Introduction

L'objectif de ce TP est de realiser une application Android qui affiche une liste de contacts dans une **ListView**, copie le contact touche dans un **EditText**, puis enregistre la modification dans la liste avec le bouton **Modifier**. Les notions abordees sont : la ListView, l'**ArrayAdapter**, la liste modifiable `mutableListOf`, `setOnItemClickListener` et `notifyDataSetChanged()`. En plus du cahier des charges de base, les cinq exercices de la section 6 du sujet ont ete traites : ajout, suppression avec confirmation, refus des doublons, tri alphabetique et conservation des donnees lors de la rotation.

2. Projet et interface

- **Projet :** modele *Empty Views Activity*, nom ContactsListView, package `com.example.contacts`, Kotlin, minSdk 24, Kotlin DSL.
- **Layout :** un LinearLayout vertical. En haut, une ligne horizontale avec le champ de saisie (largeur 0dp, poids 1, qui occupe tout l'espace laisse par les boutons) et les boutons *Ajouter* et *Modifier*. En dessous, la ListView (hauteur 0dp, poids 1) occupe le reste de l'ecran, avec `choiceMode="singleChoice"` pour memoriser un seul element selectionne.
- **Ressources :** aucun texte en dur dans le layout. Les libelles et messages sont dans `strings.xml` (l'apostrophe est echappee avec un antislash) et les couleurs dans `colors.xml`.
- **Presentation :** chaque ligne utilise une mise en page personnalisee (`item_contact.xml`) avec une icone et le nom ; la ligne selectionnee est mise en surbrillance. Le champ est un champ Material avec le libelle *Nom du contact*.

3. Fonctionnement

3.1 Les donnees et l'adaptateur

Les contacts sont stockes dans une liste **modifiable** (`mutableListOf`) : avec `listOf`, la liste serait en lecture seule et l'instruction `contacts[i] = ...` ne compilerait pas. La ListView ne sait pas afficher directement une liste Kotlin : l'ArrayAdapter transforme chaque chaine en une vue. Le schema est donc : *contacts (MutableList) vers ArrayAdapter vers ListView*.

3.2 Selection d'un contact

Au clic sur une ligne, `setOnItemClickListener` memorise la position dans `positionSelectionnee` (-1 signifie aucune selection), coche la ligne avec `setItemChecked` et copie le nom dans le champ. Le lambda recoit quatre parametres (parent, vue, position, identifiant) ; seule la position est utilisee, les autres sont remplaces par _.

3.3 Modification

Avant de modifier, deux controles sont faits : si aucun contact n'est selectionne, un **Toast** le signale ; si le texte est vide, une erreur est affichee sur le champ (`etContact.error`). Ensuite la donnee est mise a jour puis l'adaptateur est prevenu avec `adapter.notifyDataSetChanged()`. C'est le point cle du TP : sans cet appel, la liste Kotlin change mais l'ecran n'est pas redessine. La touche *Termine* du clavier valide aussi la modification (bonus du sujet).

4. Exercices complementaires realises

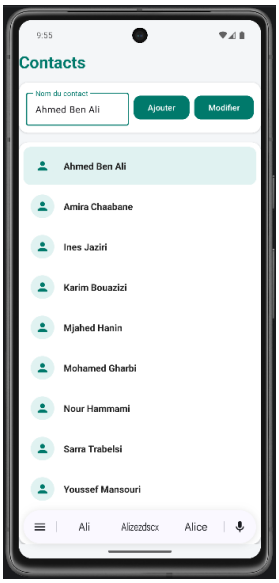
- **Ajouter :** un bouton *Ajouter* insere le texte du champ comme nouveau contact (`contacts.add(...)`), puis un message confirme l'ajout (par exemple "Contact ajoute : Mestaoui Habiba").
- **Supprimer :** un appui long (`setOnItemLongClickListener`) ouvre un **AlertDialog** de confirmation ("Voulez-vous vraiment supprimer ... ?" avec les boutons Non et Oui). Le contact n'est retire qu'apres confirmation.
- **Doublons :** si le nom existe deja dans la liste, l'ajout ou la modification est refuse et le champ affiche "Ce contact existe deja dans la liste".
- **Tri :** la liste est triee par ordre alphabetique apres chaque modification (`contacts.sort()`), ce qui se voit sur les captures.
- **Rotation :** les donnees sont conservees dans un **ViewModel** (classe ContactsViewModel), voir la reponse a la question de reflexion.

5. Tests

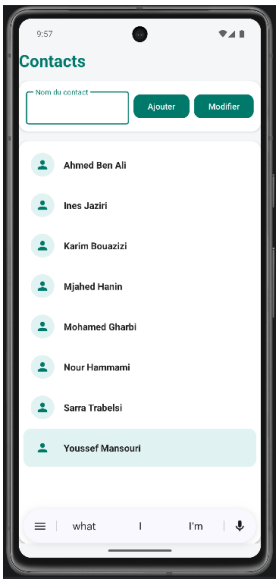
L'application a ete executee sur l'emulateur Pixel 7 (API 36). Chaque scenario ci-dessous est illustre par une capture (dossier screenshots du depot).

N	Scenario	Resultat observe
1	Cliquer sur un contact	Ligne surlignee, nom copie dans le champ
2	Ajouter un contact	Contact insere, liste triee, message de confirmation
3	Modifier un contact	Liste mise a jour, message "Contact modifie"
4	Saisir un nom deja present	Refus, message "Ce contact existe deja dans la liste"
5	Appui long sur un contact	Boite de dialogue de confirmation (Non / Oui)
6	Confirmer la suppression	Contact retire de la liste, message "Contact supprime"

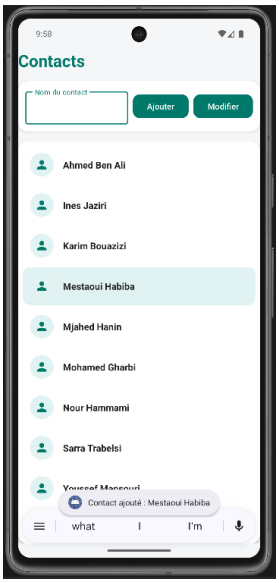
Captures d'ecran



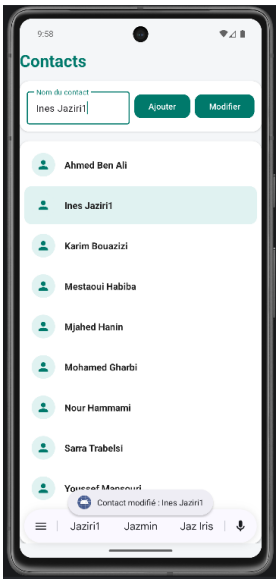
1. Liste et contact selectionne



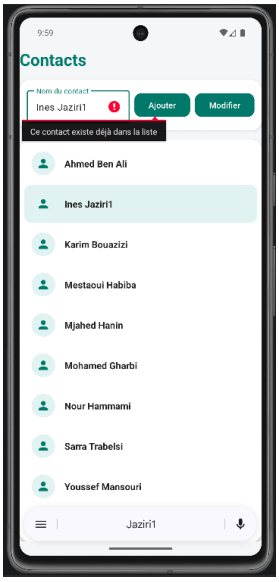
2. Etat apres suppression



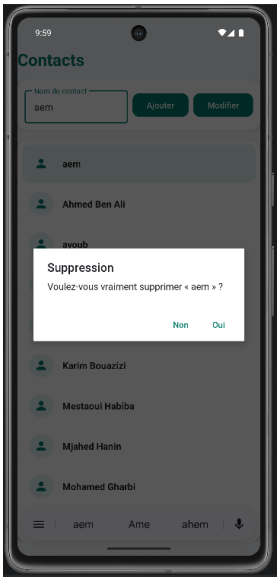
3. Ajout d'un contact



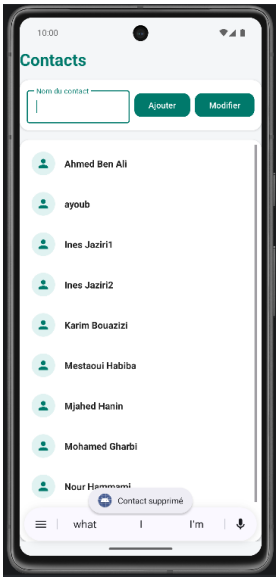
4. Modification d'un contact



5. Doublon refuse



6. Confirmation de suppression



7. Suppression effectuee

6. Questions de reflexion

Pourquoi mutableListOf et pas listOf ?

`listOf` cree une liste en lecture seule. Comme l'application doit modifier, ajouter et supprimer des elements, il faut une liste modifiable : `mutableListOf`.

Que se passe-t-il quand on tourne l'ecran ?

A la rotation, Android detruit puis recree l'activite pour appliquer la nouvelle configuration. Les variables de l'activite, dont la liste des contacts, sont reinitialisees : les modifications sont donc perdues et la liste d'origine reapparait. La solution est de placer les donnees dans un **ViewModel**, qui survit aux changements de configuration : la nouvelle activite recupere la meme liste. Un ViewModel ne survit toutefois pas a la mort du processus ; pour des donnees durables, il faudrait une sauvegarde persistante.

Role de notifyDataSetChanged()

L'adaptateur ne surveille pas la liste Kotlin. Apres chaque changement des donnees, notifyDataSetChanged() l'informe qu'il doit redessiner la ListView.

7. Difficultes rencontrees

- **Depot Git cree dans le mauvais dossier.** Les commandes ont d'abord ete lancees depuis le sous-dossier screenshots, ce qui a cree un depot au mauvais endroit. Il a fallu supprimer ce depot, remonter a la racine du projet, puis refaire *git init* et les commits.
- **Nom du depot distant.** Le depot GitHub doit exister avec exactement le meme nom que dans l'adresse du *remote* avant le premier *push*, sinon Git repond "Repository not found".

8. Conclusion

Ce TP a permis de comprendre la chaine donnees, adaptateur et vue d'une ListView, de reagir aux clics, et surtout de retenir que modifier la liste ne suffit pas : il faut prevenir l'adaptateur. Les exercices complementaires (ajout, suppression confirmee, doublons, tri) et la conservation des donnees avec un ViewModel completent le cahier des charges de base.